



DSA612S Distributed Systems and Applications | Assignment 2

Namibia Eats

Distributed Food Delivery Platform - Project Documentation, Work Allocation and Defence Guide

Item	Details
Course	Distributed Systems and Applications (DSA612S), Namibia University of Science and Technology
Group	Group 22 - 8 members
Group leader	Haufiku Frans (222127147)
Repository	https://github.com/fransrichy/Distributive-System-2026-Assignment-Two
Due	05 October 2026, 23:59 (commits after this time are not accepted)
Stack	Ballerina 2201.13.6, Apache Kafka 3.9 (KRaft), MongoDB 7, Docker Compose, nginx, Prometheus, Grafana

Group members

#	Student number	Full name	Area of responsibility
1	222127147	Haufiku Frans	Group leader - integration, Docker orchestration, repository, demo
2	225053241	Eliaser Angula	Order Service (state machine and saga)
3	225156237	Matatias Nghihangwa	Payment Service
4	225123851	Pandera Katjipuka	Restaurant Service
5	222075449	Monika Shalauda	Customer Service
6	225042428	Risto Sakeus	Delivery Service (routing and driver simulation)
7	223003018	Alanray Miller	Notification Service and Kafka topic management
8	225017288	Kavara Edward	Admin Service, observability (Prometheus/Grafana) and web UI

Contents

Group members.....	1
1. Introduction	3
1.1 Objectives covered.....	3
2. System architecture	3
2.1 Component view	3
2.2 The seven services.....	3
2.3 Order lifecycle (state machine)	4
2.4 Order saga (end-to-end flow)	4
2.5 Failure handling and compensation.....	5
3. Kafka setup and topic management.....	5
3.1 Producer and consumer logic	6
4. Database setup and schema design	6
5. Docker configuration and orchestration	7
6. Bonus features.....	7
7. How to run the platform	7
7.1 Prerequisites (Windows)	7
7.2 Start, use and stop	7
8. Testing	8
9. Work allocation	9
9.1 Git workflow for every member.....	10
9.2 Timeline for 05 October 2026.....	10
10. Live defence plan	10
10.1 Demonstration script (about 10 minutes)	10
10.2 Likely questions and model answers.....	11
11. Repository structure.....	11

1. Introduction

The Ministry of Industrialisation and Trade requested a distributed platform that supports local SMEs - restaurants and delivery drivers. When a customer places an order the restaurant must be notified immediately, the payment must be processed securely and a driver must be dispatched based on availability. The platform must cope with high concurrency at peak meal times and remain fault-tolerant.

Namibia Eats answers this with seven independent Ballerina microservices that coordinate through Kafka events. Each service owns its own MongoDB database, everything runs in Docker containers and Docker Compose starts the complete environment with one command.

1.1 Objectives covered

Learning objective	How the project demonstrates it
Microservices with clear boundaries (Ballerina)	Seven services, each with its own REST API, database and Kafka consumer group. Pure business logic lives in a separate domain module per service.
Event-driven design with Kafka	17 explicitly created topics, keyed partitions, idempotent producers, consumer groups, manual offset commits, retries and a dead-letter topic.
Persistence of relational and event data	MongoDB database-per-service with \$jsonSchema validation, unique indexes for idempotency and event-sourced read models.
Containerisation and orchestration	Multi-stage Dockerfiles, non-root containers, health checks, dependency ordering, isolated networks, memory limits and scaling.

2. System architecture

The platform follows a **microservice, event-driven architecture**. Services never read each other databases. State changes travel as Kafka events; synchronous REST calls are used only for read-only lookups (for example the Order Service reading a menu to price an order).

2.1 Component view



2.2 The seven services

Service	Port	Database	Responsibility
Customer	8081	customer_db	User accounts (bcrypt passwords), delivery addresses with one default, notification preferences, historical order data built from events.
Restaurant	8082	restaurant_db	Restaurants, digital menus, real-time inventory (atomic stock reservation), kitchen opening hours (overnight-aware), kitchen ticket queue.
Order	8083	order_db	Central order state machine and lifecycle, saga coordination, surge pricing, circuit-breaker protected calls to other services.

Service	Port	Database	Responsibility
Payment	8084	payment_db	Simulated payment gateway (card, mobile money, cash on delivery), idempotent charging, refunds and voids, confirmation events.
Delivery	8085	delivery_db	Driver management, ETA-based driver assignment, A* route optimisation, real-time location simulation and delivery tracking.
Notification	8086	notification_db	Multi-channel alerts (email, SMS, push) to customers, restaurants and drivers, honouring customer preferences, de-duplicated per event.
Admin	8087	admin_db	Reports on restaurant statistics and delivery performance, platform KPIs, live fleet, Kafka topic counters, dead-letter queue, system health.

2.3 Order lifecycle (state machine)

The Order Service owns the lifecycle CREATED -> CONFIRMED -> PREPARING -> READY -> OUT_FOR_DELIVERY -> DELIVERED, with CANCELLED possible from CREATED and CONFIRMED.

From	To	Triggered by	Produced by
(new)	CREATED	POST /orders after validating customer, menu, stock and opening hours	Order Service (REST)
CREATED	CONFIRMED	payments.completed	Payment Service
CREATED	CANCELLED	payments.failed or customer cancellation	Payment Service / customer
CONFIRMED	PREPARING	restaurant.order-preparing	Restaurant Service
CONFIRMED	CANCELLED	restaurant.order-rejected (sold out) or customer cancellation	Restaurant Service / customer
PREPARING	READY	restaurant.order-ready	Restaurant Service
READY	OUT_FOR_DELIVERY	delivery.picked-up	Delivery Service
OUT_FOR_DELIVERY	DELIVERED	delivery.completed	Delivery Service

Three properties make the state machine safe in a distributed system:

- **Idempotent** - a duplicate or stale event produces an empty transition plan and is ignored.
- **Out-of-order tolerant** - if delivery.picked-up is seen before restaurant.order-ready (different topics), the order walks forward through READY so the history stays complete.
- **Concurrency safe** - every update is a compare-and-set on a version field (optimistic locking), so a customer cancellation and a Kafka event can never overwrite each other.

2.4 Order saga (end-to-end flow)

1. Customer calls POST /orders. The Order Service validates the customer (Customer Service), the restaurant opening hours and the menu stock (Restaurant Service), computes the delivery fee with surge pricing, stores the order as CREATED and publishes orders.created.
2. The Payment Service consumes orders.created, charges the customer (once per order) and publishes payments.completed or payments.failed.
3. The Order Service moves the order to CONFIRMED and publishes orders.confirmed.
4. In parallel: the Restaurant Service reserves the stock and queues a kitchen ticket; the Delivery Service plans an A* route for every free driver and assigns the fastest (delivery.assigned).

5. The kitchen starts and finishes cooking (restaurant.order-preparing, restaurant.order-ready); the order moves to PREPARING and READY.
6. The driver collects the food (delivery.picked-up -> OUT_FOR_DELIVERY), drives along the optimised route publishing delivery.location-updated, and hands it over (delivery.completed -> DELIVERED).
7. The Notification Service turns every customer-facing event into email/SMS/push alerts, and the Admin Service turns every event into analytics.

2.5 Failure handling and compensation

Failure	What the platform does
Card declined (test card ending 0000) or wallet limit	payments.failed -> order CANCELLED -> customer receives SMS and push.
Stock sold out between ordering and confirmation	Restaurant releases the lines it already reserved, publishes restaurant.order-rejected -> order CANCELLED -> orders.cancelled -> Payment refunds.
Customer cancels (CREATED or CONFIRMED)	orders.cancelled -> Payment refunds (or voids if the charge had not happened yet), Restaurant releases stock, Delivery releases the driver.
Kafka unavailable while ordering	The new order is cancelled immediately and the API answers 503 - no orphan orders.
Event cannot be processed	Retried 3 times with back-off, then moved to events.dlq; the partition is never blocked.
Delivery Service down	Circuit breaker opens; surge pricing falls back to x1.00; queued deliveries are assigned when drivers return.
No free driver	Delivery waits in PENDING_ASSIGNMENT and is retried every simulation tick.

3. Kafka setup and topic management

Kafka runs in **KRaft mode** (no ZooKeeper). Automatic topic creation is switched off; the kafka-init container runs infra/kafka/create-topics.sh, which creates every topic with an explicit partition count and retention period before any service starts.

Topic	Part.	Key	Producer	Consumers
customers.registered	3	customerId	Customer	Notification, Admin
orders.created	6	orderId	Order	Payment, Admin
orders.confirmed	6	orderId	Order	Restaurant, Delivery, Admin
orders.status-changed	6	orderId	Order	Customer, Notification, Admin
orders.cancelled	3	orderId	Order	Payment, Restaurant, Delivery, Admin
payments.completed	6	orderId	Payment	Order, Notification, Admin
payments.failed	3	orderId	Payment	Order, Notification, Admin
payments.refunded	3	orderId	Payment	Notification, Admin
restaurant.order-preparing	3	orderId	Restaurant	Order, Admin
restaurant.order-ready	3	orderId	Restaurant	Order, Delivery, Admin
restaurant.order-rejected	3	orderId	Restaurant	Order, Admin
delivery.assigned	3	orderId	Delivery	Order, Notification, Admin

Topic	Part.	Key	Producer	Consumers
delivery.picked-up	3	orderId	Delivery	Order, Admin
delivery.location-updated	6	driverId	Delivery	Admin (live fleet) - 1 hour retention
delivery.completed	3	orderId	Delivery	Order, Notification, Admin
notifications.sent	3	orderId	Notification	Admin
events.dlq	1	source topic	All services	Admin (dead-letter viewer) - 14 days retention

3.1 Producer and consumer logic

- **Partitioning:** every order event is keyed by orderId, so all events of one order land in the same partition and are consumed in order, while different orders are processed in parallel. Driver telemetry is keyed by driverId.
- **Producers:** acks=all and enable.idempotence=true - producer retries can never create duplicates on the broker.
- **Consumers:** one consumer group per service, autoCommit=false; offsets are committed only after the batch was handled (at-least-once delivery).
- **Idempotent handlers:** unique indexes and state checks turn at-least-once delivery into an effectively-once result.
- **Envelope:** every message has eventId, eventType, source, occurredAt, key, schemaVersion, data.
- **Scaling:** hot topics have 6 partitions, so docker compose up -d --scale payment-service=3 shares the load and Kafka rebalances automatically.

4. Database setup and schema design

MongoDB is used with the **database-per-service** pattern. infra/mongo/init-mongo.js creates every collection with a \$jsonSchema validator and the indexes required by the access patterns.

Database	Collections	Modelling decisions
customer_db	customers, order_history	Addresses are embedded (always read with the customer). Unique email. Order history is a read model fed by orders.status-changed.
restaurant_db	restaurants, menu_items, kitchen_tickets	One document per dish so stock can be decremented atomically with stock >= quantity; the validator forbids negative stock. One ticket per order (unique).
order_db	orders	An order is one aggregate: items, address and status history are embedded; prices are copied at order time; version enables optimistic locking.
payment_db	payments	Unique index on orderId = at most one charge per order. Only the last four card digits are stored.
delivery_db	drivers, deliveries	A delivery stores its current optimised route, progress and live position; unique orderId.
notification_db	notifications, contacts	Deterministic id eventId-recipient-channel with a unique index - a redelivered event never sends the same alert twice.
admin_db	order_facts, event_counters, processed_events, fleet_positions, dead_letters	Analytics read model (CQRS). processed_events gives exactly-once effects for reports.

5. Docker configuration and orchestration

- **Multi-stage images:** the ballerina/ballerina:2201.13.6 image compiles each service; a slim eclipse-temurin:21-jre image runs it as a **non-root** user with bounded JVM memory.
- **Health checks:** every service exposes /health; Docker health checks run against it. Kafka and MongoDB have their own health checks.
- **Start-up ordering:** services start only after MongoDB is healthy and kafka-init has completed (service_healthy, service_completed_successfully).
- **Isolation:** Kafka and MongoDB live only on the backend network; the gateway lives only on frontend. Clients can only reach the services through the gateway.
- **Stability:** restart policies, per-container memory limits, log rotation.
- **Profiles:** monitoring adds Prometheus and Grafana; tools adds Kafka UI.

6. Bonus features

Bonus item	Implementation
Driver location simulation	Drivers move along their route every second and publish delivery.location-updated. The web UI shows the driver moving on an OpenStreetMap (Leaflet) overlay with an ETA.
Route optimisation	A* search over a Windhoek road-network graph (arterial 60 km/h, local 35 km/h, CBD congestion at peak). Every free driver is routed to the restaurant and the fastest ETA wins.
Surge pricing	Delivery fee = (N\$15 + N\$4 per km) x surge. Surge rises with demand per available driver, fleet utilisation and lunch/dinner peaks, capped at x2.5.
Complete UI	http://localhost:8080 - Customer (order and live tracking), Restaurant (kitchen board, inventory), Driver (online/offline, pickup, deliver) and Admin (KPIs, statistics, fleet map, health) views.
Observability	Prometheus scrapes HTTP and custom business metrics from every service; a provisioned Grafana dashboard shows orders, transitions, Kafka throughput, payments and drivers.

7. How to run the platform

7.1 Prerequisites (Windows)

1. Install Docker Desktop (WSL 2 backend). In an administrator PowerShell run wsl --install --no-distribution and restart the PC.
2. Create C:\Users\<you>\wslconfig with [wsl2], memory=3GB, swap=4GB so Docker has enough memory.
3. Open Docker Desktop and wait for **Engine running**. Quit XAMPP and other heavy programs.

7.2 Start, use and stop

```
git clone https://github.com/fransrichy/Distributive-System-2026-Assignment-Two.git
cd Distributive-System-2026-Assignment-Two
docker compose up -d --build           # core platform + web UI
docker compose --profile monitoring up -d --build # + Prometheus and Grafana
docker compose ps                       # wait until services are healthy
powershell -ExecutionPolicy Bypass -File scripts\demo.ps1 # scripted end-to-end demo
docker compose down                    # stop (add -v to delete data)
```

What	Address
Web UI	http://localhost:8080
API gateway	http://localhost:8080/api/<service>-service/...
Grafana (admin / admin)	http://localhost:3000

What	Address
Prometheus	http://localhost:9090
Kafka UI (profile tools)	http://localhost:8090
Demo customer login	demo@fooddelivery.na / password123

8. Testing

- **Unit tests:** 43 tests over the pure domain logic - state machine, surge pricing, opening hours, payment rules, A* routing, notification routing and report calculations. Run bal test inside each service folder.
- **End-to-end:** scripts/demo.ps1 places real orders through the gateway and follows the happy path, a declined card and a cancellation with refund.
- **Resilience checks:** stop a service (docker compose stop delivery-service) and show that orders still work and surge degrades gracefully; scale a consumer (--scale payment-service=3) and show Kafka rebalancing.

9. Work allocation

Every member owns one area, makes **their own changes** in it, commits them **from their own GitHub account** and is ready to defend it. The assignment states that a member whose username does not appear in the commit log receives 0, so every person below must push at least the commits listed.

Rule for everyone: only commit work you did and understand. Read your service first, run it, then make your change. You must be able to explain every line you commit during the defence.

Member	Area	Work to do and commit today	Must be able to explain
Haufiku Frans 222127147 Group leader	Integration, Docker Compose, repository, demo	1. Invite all members as collaborators and protect main. 2. Review and merge every pull request. 3. Run the full stack and the demo script; add screenshots to docs/screenshots/. 4. Fill in the group table in README.md.	docker-compose.yml: networks, health checks, depends_on, profiles, memory limits; API gateway; overall architecture.
Eliaser Angula 225053241	Order Service	1. Add endpoint GET /orders/{orderId}/history returning only the status history. 2. Add two unit tests to modules/domain/tests (e.g. PREPARING -> DELIVERED catch-up; customer may cancel in CREATED). 3. Write docs/services/order.md.	State machine, planTransition, optimistic locking with version, saga steps, surge pricing call.
Matatias Nghihangwa 225156237	Payment Service	1. Add endpoint GET /payments/stats/by-method (count and amount per payment method). 2. Add boundary unit tests (exactly N\$5000 by card is approved; N\$1500.01 cash is declined). 3. Write docs/services/payment.md.	Idempotent charging (unique orderId), refund versus void, why only last 4 digits are stored.
Pandera Katjipuka 225123851	Restaurant Service	1. Add one new restaurant with 4 menu items to the seed data. 2. Add a unit test for a schedule that closes exactly at midnight (18:00-00:00). 3. Write docs/services/restaurant.md.	Atomic stock reservation and rollback, opening hours over midnight, kitchen ticket statuses.
Monika Shalauda 222075449	Customer Service	1. Add endpoint PUT /customers/{id}/password (verify old password with bcrypt, store the new hash). 2. Test it with Postman/curl and add the request to docs/api.md. 3. Write docs/services/customer.md.	Accounts and bcrypt, embedded addresses and the single-default rule, order history read model.
Risto Sakeus 225042428	Delivery Service	1. Add endpoint GET /drivers/{id}/earnings (deliveries, distance, earnings). 2. Add a unit test for rankDrivers with three drivers. 3. Write docs/services/delivery.md.	A* algorithm and heuristic, driver assignment with compare-and-set, location simulation, ETA.
Alanray Miller 223003018	Notification Service + Kafka	1. Add endpoint GET /notifications/unread-count?recipientType=&recipientId=. 2. Add a comment block in infra/kafka/create-topics.sh explaining the partition count of each hot topic. 3. Write docs/services/notification.md.	Topics, partitions and keys, consumer groups, manual commits, retries and DLQ, notification de-duplication.

Member	Area	Work to do and commit today	Must be able to explain
Kavara Edward 225017288	Admin Service, monitoring, UI	1. Add endpoint GET /reports/cancellations (count per cancel reason). 2. Add one Grafana panel (payments per minute) to infra/grafana/dashboards/food-delivery.json. 3. Write docs/services/admin.md.	CQRS read models, processed_events idempotency, Prometheus scraping, Grafana dashboard, web UI views.

9.1 Git workflow for every member

1. Set your identity once (use the same email as your GitHub account):

```
git config --global user.name "Your Full Name"
git config --global user.email "your-github-email@example.com"
```

2. Clone the repository and create a branch for your work:

```
git clone https://github.com/fransrichy/Distributive-System-2026-Assignment-Two.git
cd Distributive-System-2026-Assignment-Two
git checkout -b feature/order-history # use your own branch name
```

3. Make your change, then build and test your service:

```
cd services/order-service
bal build
bal test
```

4. Commit with a clear message and push:

```
git add services/order-service docs/services/order.md
git commit -m "Order service: add GET /orders/{id}/history endpoint and tests"
git push -u origin feature/order-history
```

5. Open a Pull Request on GitHub into main. Frans reviews and merges it.
6. Before starting new work: git checkout main and git pull.

9.2 Timeline for 05 October 2026

Time	Who	Activity
08:00 - 09:30	Everyone	Clone the repository, install Docker/Ballerina, run the platform, place an order in the UI.
09:30 - 13:00	Everyone	Read your service, make your change and tests, write your service notes.
13:00 - 16:00	Everyone / Frans	Push branches, open pull requests; Frans reviews and merges.
16:00 - 18:00	Frans + 2 volunteers	Full rebuild from main, run scripts/demo.ps1, take screenshots, fix anything broken.
18:00 - 20:00	Everyone	Defence rehearsal - each member explains their area in 2 minutes and answers questions.
by 21:00	Frans	Final push. Keep a safety margin - commits after 23:59 are not accepted.

10. Live defence plan

10.1 Demonstration script (about 10 minutes)

1. **Frans:** show docker compose ps (all healthy), explain the architecture diagram and the networks.
2. **Monika:** log in as the demo customer, show addresses and order history.
3. **Pandera:** open the Restaurant view - menu, live stock, opening hours, kitchen board.

4. **Eliaser:** place an order and follow the state machine in the tracking stepper.
5. **Matatias:** show the payment record; place a second order with card 0000 and show it being cancelled.
6. **Risto:** show the driver moving on the map along the A* route and explain the ETA.
7. **Alanray:** show the notifications that were sent and list the Kafka topics (kafka-topics.sh --describe).
8. **Kavara:** show the Admin dashboard and Grafana, then stop a service and show the health change.

10.2 Likely questions and model answers

Question	Short answer
Why Kafka instead of REST calls between services?	Asynchronous, decoupled communication: a slow or failed service does not block the others, events are stored durably and can be replayed, and several services can react to one event.
How do you keep the events of one order in order?	The message key is the orderId. Kafka sends all messages with the same key to the same partition, and a partition is consumed in order.
What happens if the same event is delivered twice?	Handlers are idempotent: unique indexes (one payment per order, one ticket per order) and the state machine ignore duplicates.
What happens if a message can never be processed?	It is retried three times, then published to events.dlq and the offset is committed, so the partition is not blocked. The Admin view lists dead letters.
Why a database per service?	Loose coupling and independent scaling/deployment; no service can break another by changing a shared schema.
How do you stop two customers buying the last item?	Stock is decremented with an atomic conditional update (stock >= quantity); only one update can succeed.
How is a race between cancel and an event prevented?	Optimistic locking: the order update only succeeds if the version is unchanged; otherwise it reloads and retries.
How is the driver chosen?	A* plans the fastest route from every available driver to the restaurant; the lowest ETA wins. The driver is claimed with a compare-and-set on status AVAILABLE.
How does surge pricing work?	Recent orders divided by available drivers, fleet utilisation and peak hours increase a multiplier (max x2.5) applied to the delivery fee.
How do the containers start in the right order?	depends_on with health conditions: Kafka healthy -> kafka-init creates topics -> services start once MongoDB is healthy and kafka-init completed.
How would you scale for peak time?	Run more replicas of a service; the consumer group shares the 6 partitions of hot topics. The gateway resolves replicas through Docker DNS.

11. Repository structure

```

docker-compose.yml      full orchestration (profiles: monitoring, tools)
services/
  customer-service/    restaurant-service/  order-service/  payment-service/
  delivery-service/    notification-service/  admin-service/
  Ballerina.toml      Config.toml  Dockerfile
  config.bal          environment configuration and subscribed topics
  api.bal             REST resources
  handlers.bal        Kafka event handlers
  events.bal          topic catalogue, event envelope, producer, consumer + DLQ
  platform.bal        HTTP listener, MongoDB client, /health
  modules/domain/     pure business logic and unit tests
infra/kafka/create-topics.sh  topic management
infra/mongo/init-mongo.js    schemas and indexes
infra/prometheus, infra/grafana  monitoring
ui/                          nginx gateway and web application
scripts/demo.ps1, demo.sh    end-to-end demonstration
docs/                        architecture, events, API reference

```